CODE CHALLENGE CHAMPIONSHIP

Mission 1

CodeChamps - Onsite Practice

Python Junior **Category**

2026

StarScreen Ticket Booth

Overview:

A cinema named **StarScreen** wants a simple app to be displayed on a touch screen at its entrance, so visitors can quickly know the ticket prices and calculate what they need to pay before reaching the cashier. The cinema offers three seat categories.

Mission:

1- Build a **Streamlit app** that contains:

- A **title** → "StarScreen Cinema" (with movie-related emojis).
- Display the following seat prices in EGP as bullet points:
    1. Standard → **90 EGP**
    2. Premium → **130 EGP**
    3. IMAX → **180 EGP**
- Add a **selectbox** to display the seat categories (Standard - Premium - IMAX).
- Add an **input field** to take the number of tickets from the user (minimum 1 and maximum 10).

2- Calculate the **total cost** based on the **selected seat category** and **number of tickets**, then create a **button** to display the total when clicked.

Bonus Part:

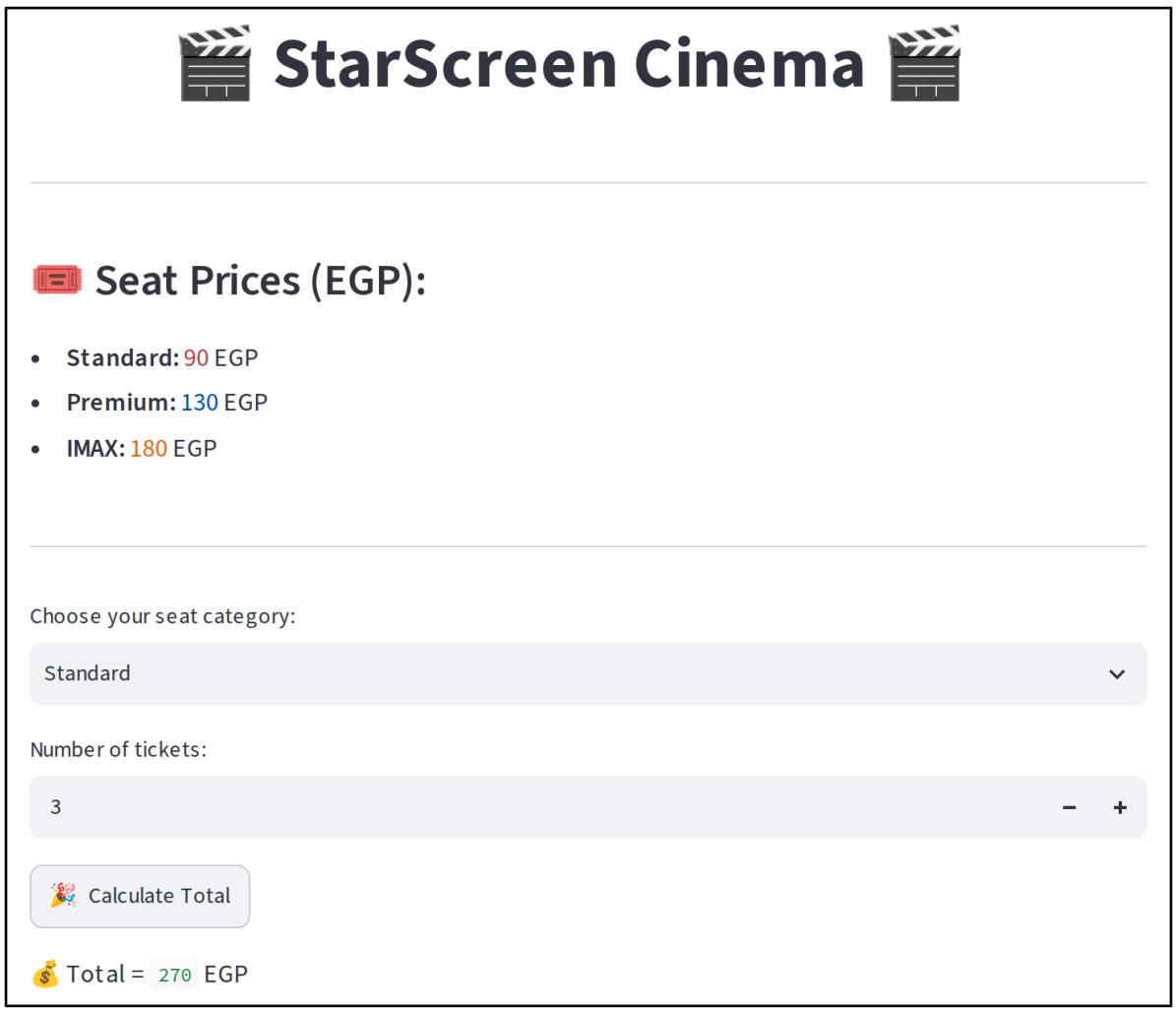
Edit the app to be in a **2-column layout** (seat category on the left, number of tickets on the right) and show the total inside a green success message.

Useful Hints:

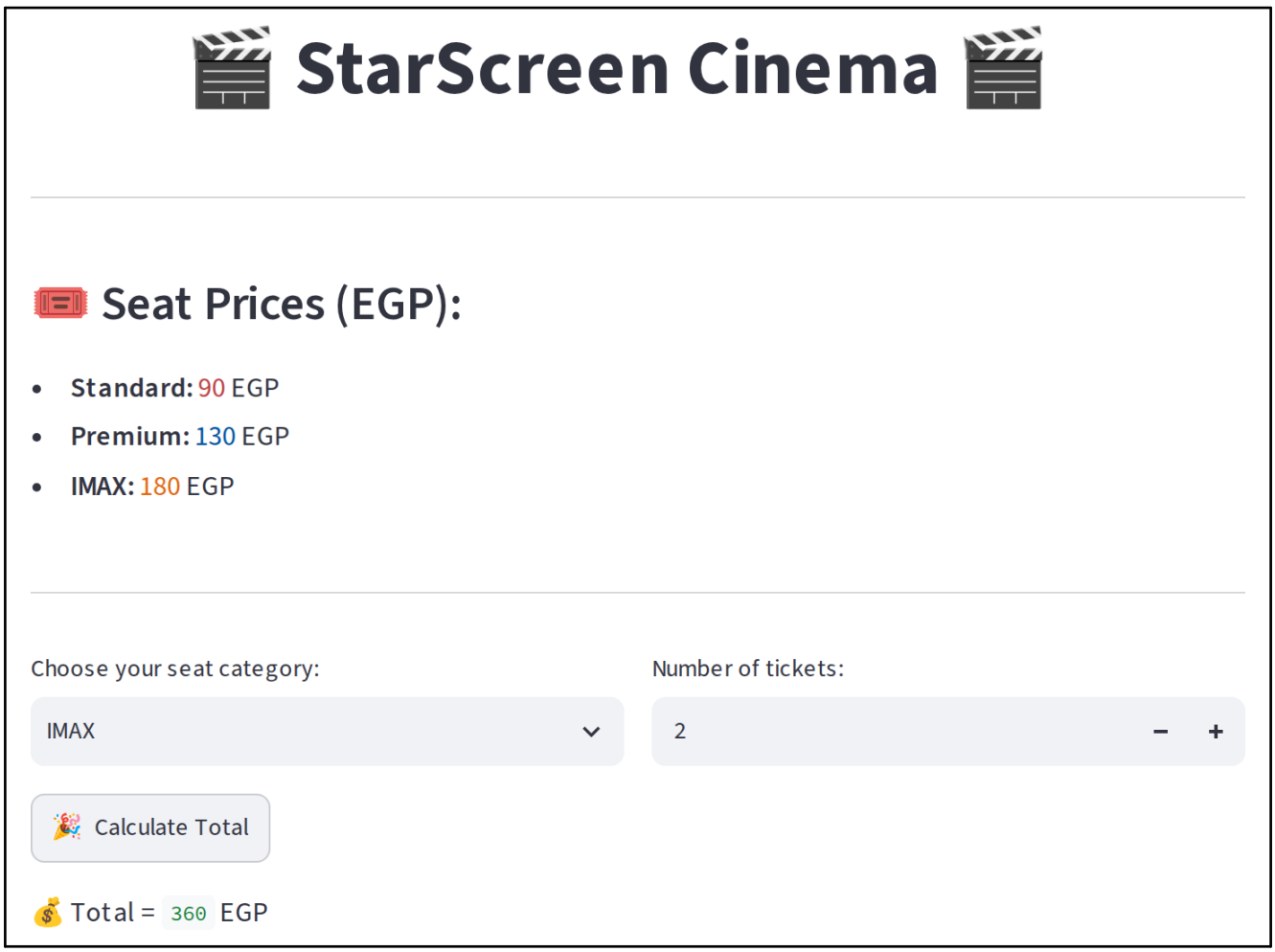
- Use **st.divider()** to make a horizontal line and space between app sections.
- Use **:color-name[Text]** to color any text. Ex: st.write(':red[90]').
- Use the **min_value** and **max_value** parameters of st.number_input to limit the tickets.
- Store the prices in a **dictionary** to get the price of the selected category easily.

Expected Output:

• **Displaying the price of 3 tickets for Standard seats:**



• **Displaying the price of 2 IMAX tickets with bonus part added:**



CODE CHALLENGE CHAMPIONSHIP

Mission 1

CodeChamps - Onsite Practice

Python Junior **Category**

2026

LevelUp Gaming Zone

Overview:

A gaming center named **LevelUp** rents gaming devices by the hour. They want a small app on the reception monitor so players can estimate the cost of their session before they start playing.

Mission:

1- Build a **Streamlit app** that contains:

- A **centered title** → "LevelUp Gaming Zone" (with game-related emojis).
- Display the following prices **per hour** in EGP as bullet points:
    1. Console → **40 EGP**
    2. PC → **30 EGP**
    3. VR → **80 EGP**
- Add a **selectbox** to choose the device type (Console - PC - VR).
- Add a **slider** to choose the number of hours (from 1 to 8 hours).

2- Calculate the **total cost** based on the **selected device** and **number of hours**, then create a **button** to display the total when clicked.

Bonus Part:

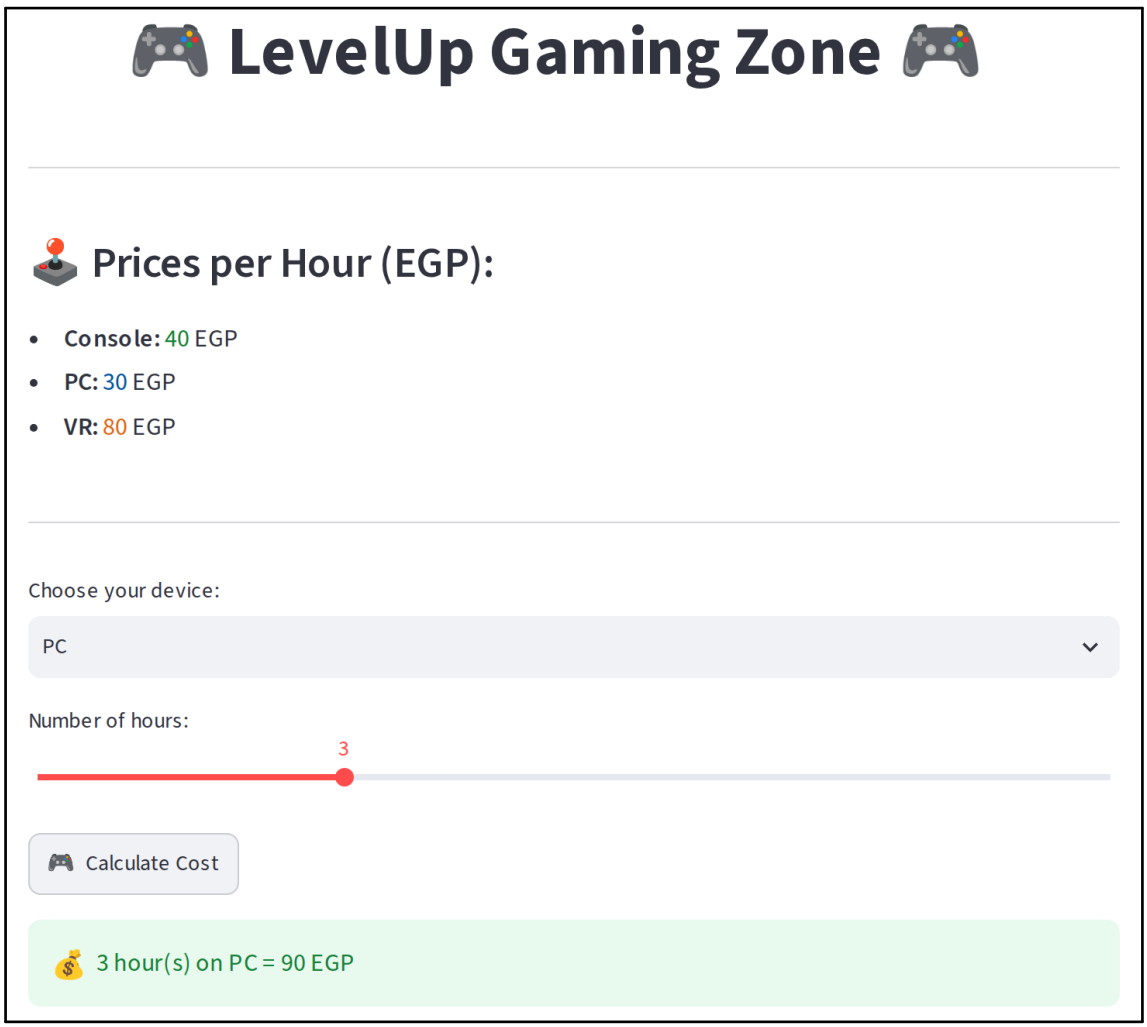
Edit the app to be in a **2-column layout** (device on the left, hours on the right) and display the total using **st.success()** with the device name and the hours included in the message.

Useful Hints:

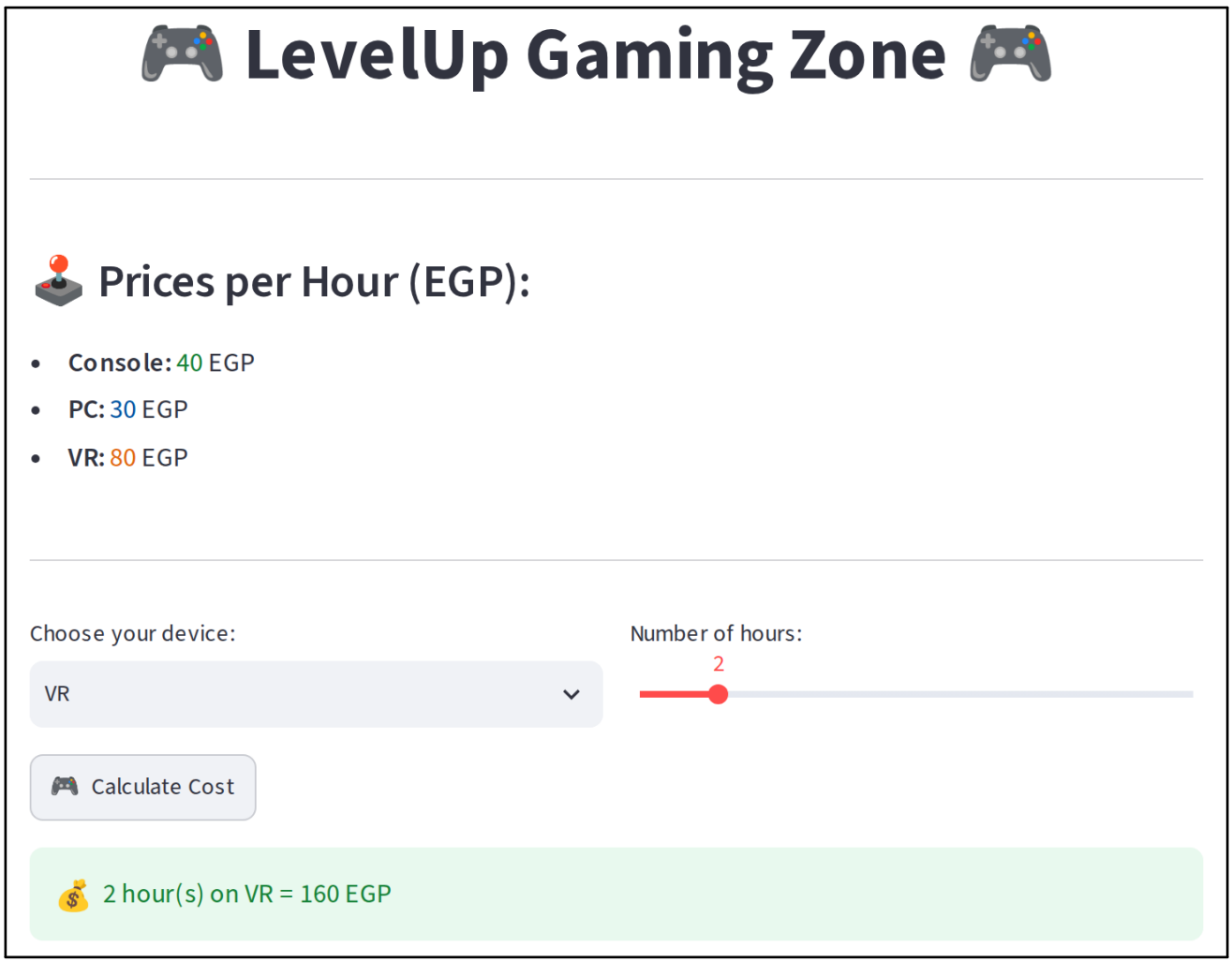
- Use **st.divider()** to separate the prices section from the inputs section.
- Use **:color-name[Text]** to color the prices. Ex: st.write(':green[40]').
- To make a **centered title**, add the **text_alignment** parameter with the value 'center'.
- Use an **f-string** to build the message shown after clicking the button.

Expected Output:

• **Displaying the price of 3 hours on a PC:**



• **Displaying the price of 2 hours on VR with bonus part added:**



CODE CHALLENGE CHAMPIONSHIP

Mission 2

CodeChamps - Onsite Practice

Python Junior **Category**

2026

University Username Generator

Overview:

The IT team in a university named **NileTech** wants to create a tool that automatically generates official student emails. The tool collects the student's information and creates the email instantly using a fixed university format, eliminating manual email creation.

Mission:

1- Create a **Streamlit** app titled by **"NileTech Student Email"** that:

- Takes the following student information:
    1. **First name** → As a **text input** with no more than **12** characters.
    2. **Last name** → As a **text input** with no more than **12** characters.
    3. **National ID** → As a **text input** with **exactly 14 digits**.
    4. **Faculty** → As a choice from a **selectbox** containing **6** different options as shown below.
- The **6 faculties available** with their codes:

Faculty	Code
Engineering	eng
Medicine	med
Business	bus
Computer Science	cs
Arts	art
Law	law

2- Create the student email based on the following format:

```
first_letter_of_first_name.last_name.faculty_code.
last_3_digits_of_ID.random_2_digit_number@NileTech.edu
```

- **Email example** → s.hassan.eng.567.41@NileTech.edu
- The email must be **lowercase** (even if the student writes the name with capital letters).

3- Add a **button** that when clicked, the email is generated instantly.

Bonus Part:

Use a **Streamlit function** that makes the generated email **easy to copy** with one click, as shown in the output section below.

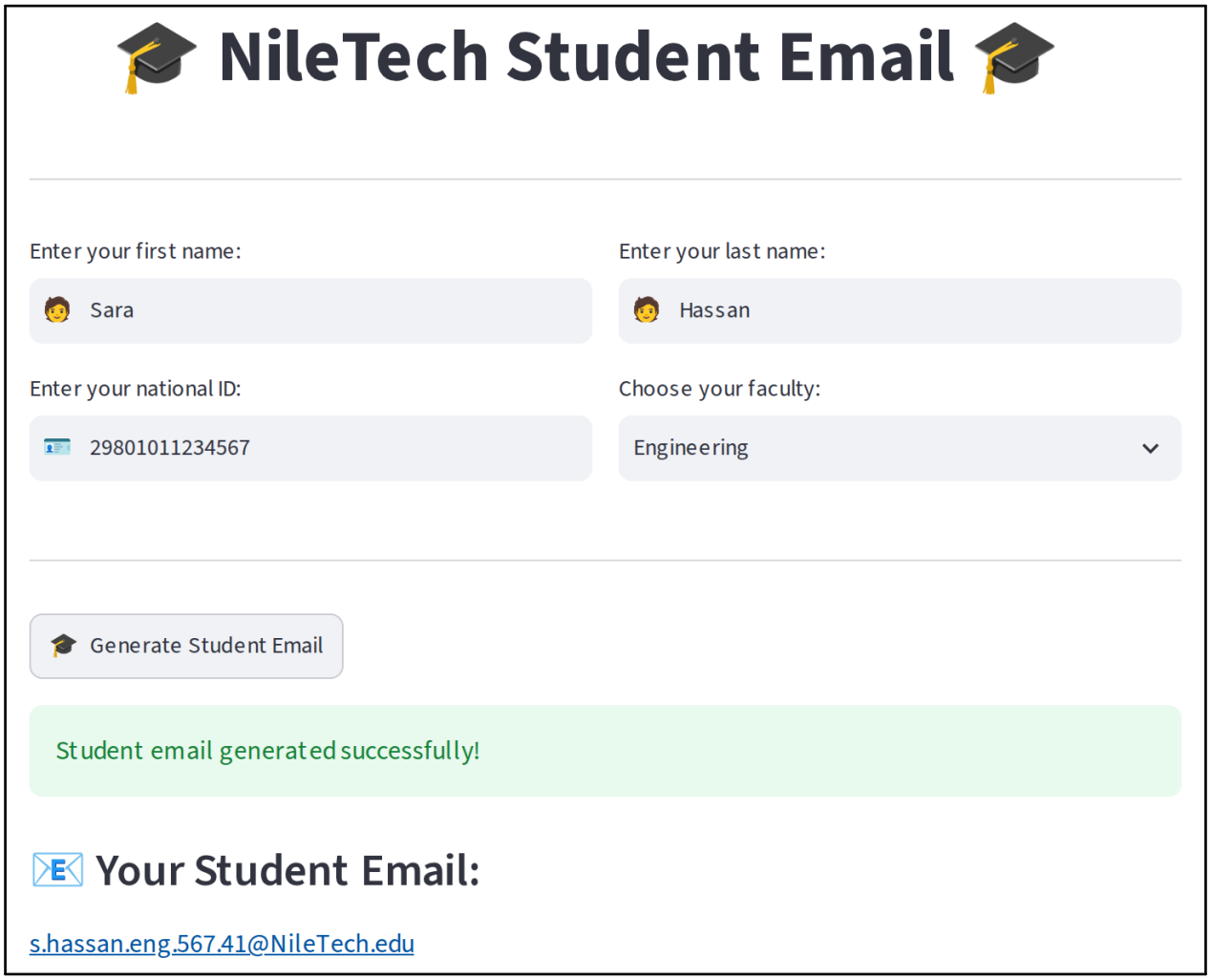
Useful Hints:

- Use the **random module** to generate the 2-digit number (from 10 to 99).
- Use **[0]** to get the first letter and **[-3:]** slicing to extract the last 3 digits of the ID.
- Use the **.lower()** string method to convert the email to lowercase.

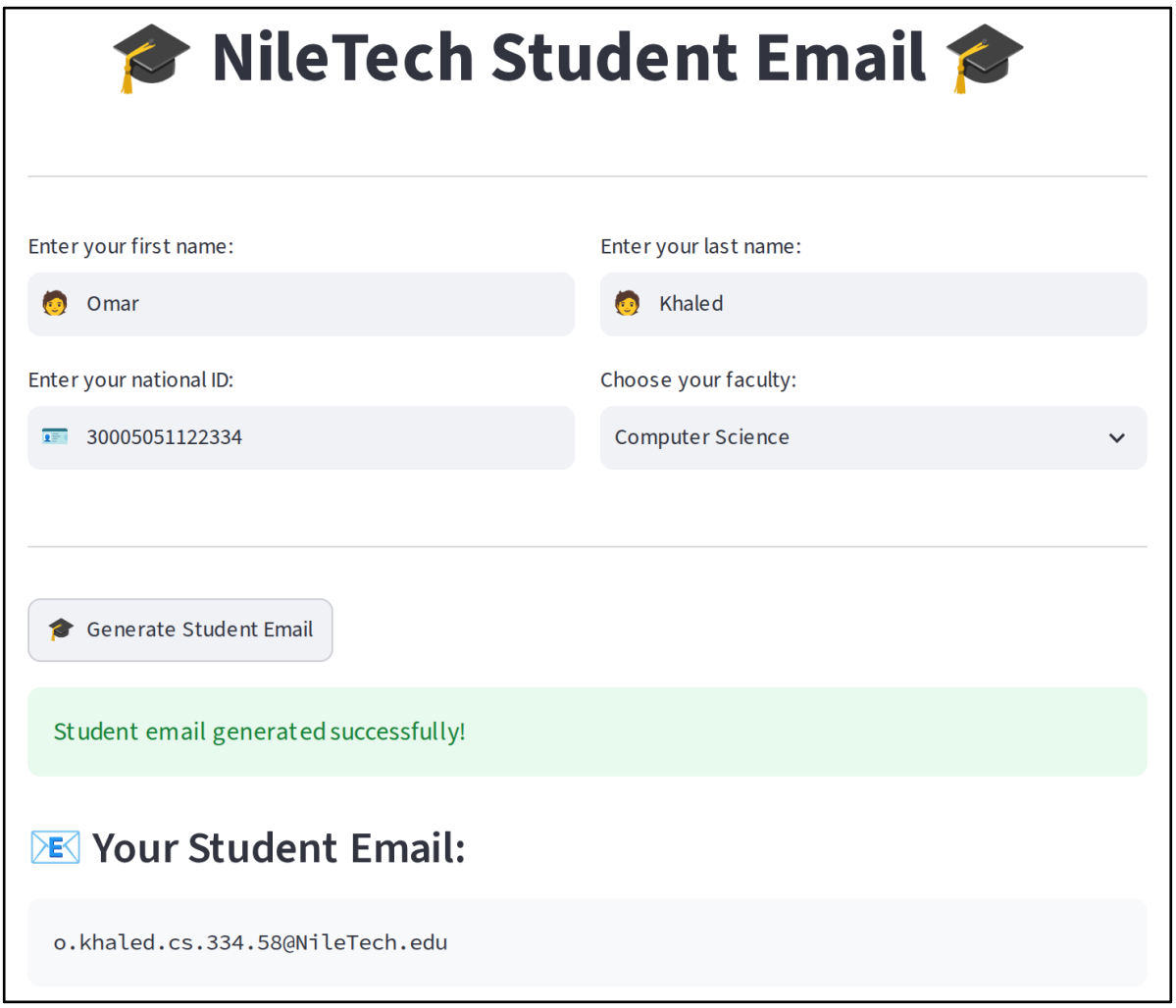
- Use the **icon parameter** to add icons to the input fields.

Expected Output:

• The main app layout without the bonus part:



• **Another example with the bonus part:**



CODE CHALLENGE CHAMPIONSHIP

Mission 2

CodeChamps - Onsite Practice

Python Junior **Category**

2026

Library Membership Code

Overview:

A public library named **ReadHub** wants to give every new member a unique membership code. The librarians need a tool that collects the member's information and creates the code instantly using a fixed library format.

Mission:

1- Create a **Streamlit** app titled by **"ReadHub Membership Center"** that:

- Takes the following member information:
    1. **Full name** → As a **text input** with no more than **15** characters (a single word).
    2. **Phone number** → As a **text input** with **exactly 11 digits**.
    3. **Membership type** → As a choice from a **selectbox** containing **5** different options as shown below.
- The **5 membership types available** with their codes:

Membership Type	Code
Student	STU
Teacher	TCH
Researcher	RSR
Senior	SNR
Guest	GST

2- Create the membership code based on the following format:

```
TYPE_CODE-FIRST_3_LETTERS_OF_NAME-LAST_4_DIGITS_OF_PHONE-RANDOM_3_DIGIT_NUMBER
```

- **Code example** → STU-LAY-7890-482
- The first 3 letters of the name must be in **UPPERCASE**.

3- Add a **button** that when clicked, the code is generated instantly. Show an **error message** instead of the code if the phone number is not exactly 11 digits.

Bonus Part:

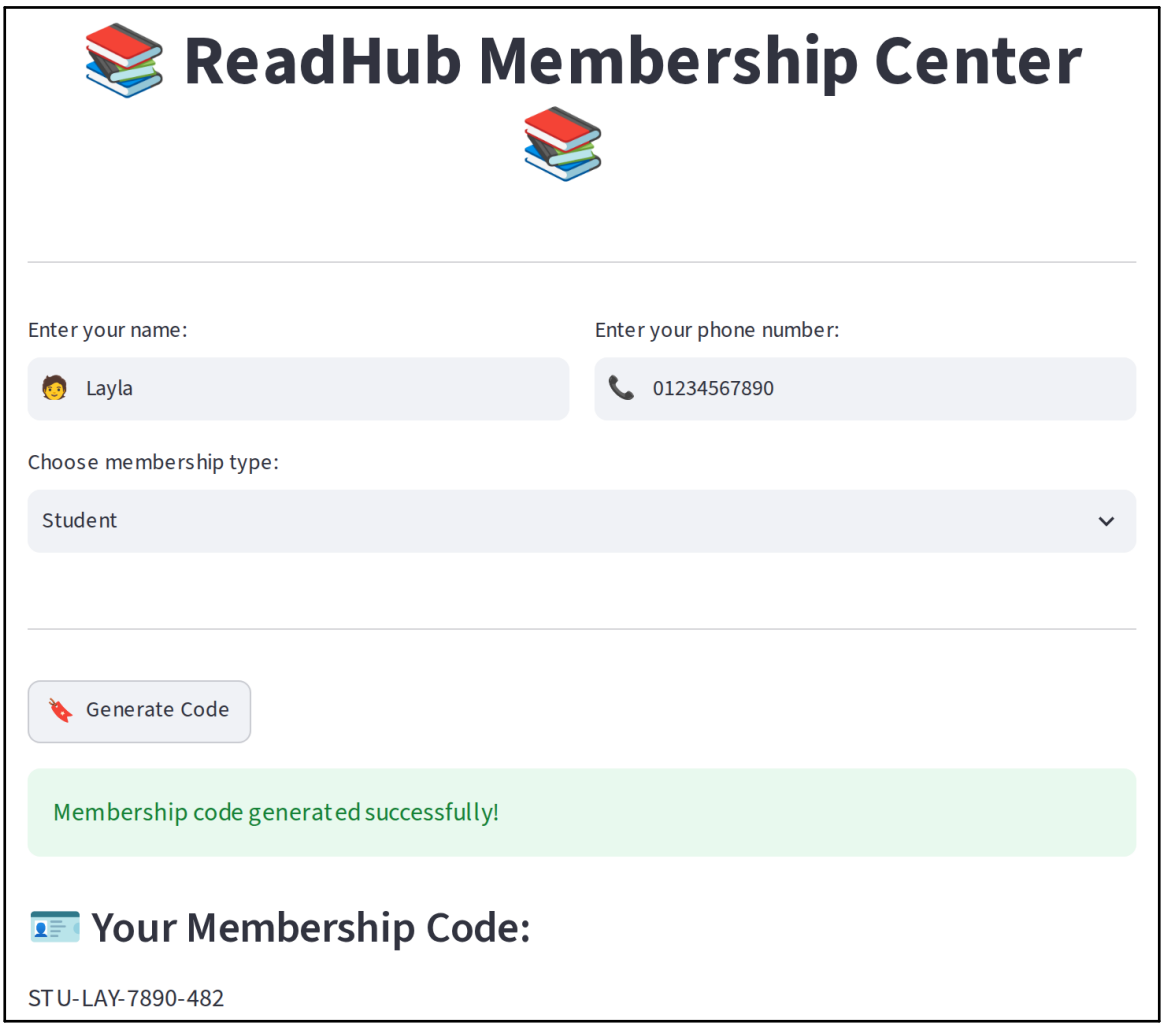
Use a **Streamlit function** that makes the generated code **easy to copy**, as shown in the output section below.

Useful Hints:

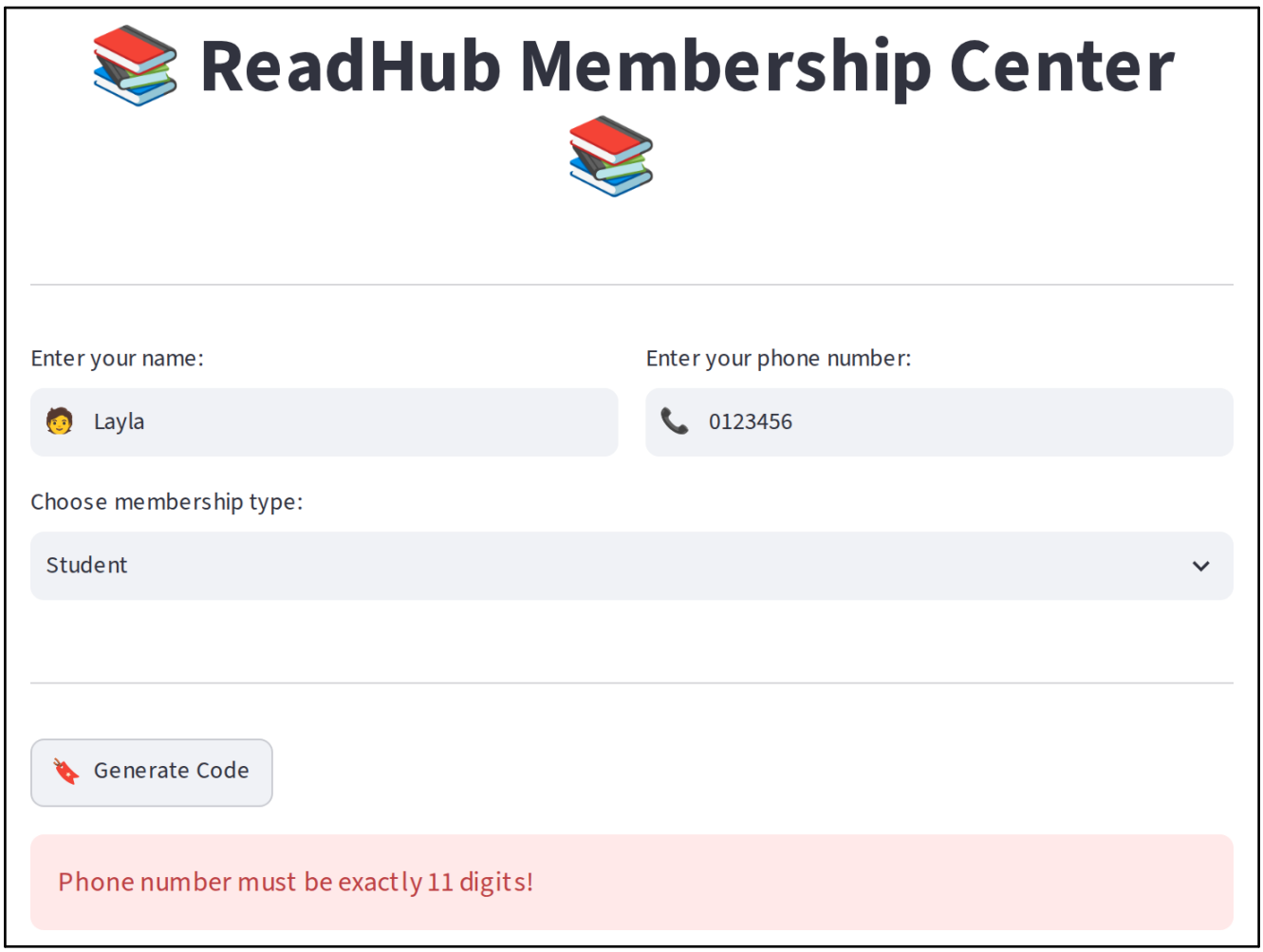
- Use the **random module** to generate the 3-digit number (from 100 to 999).
- Use **[:3]** slicing for the first 3 letters and **[-4:]** slicing for the last 4 digits.
- Use **.upper()** to convert the letters to uppercase and **.isdigit()** / **len()** to validate the phone.
- Use a **dictionary** to connect each membership type with its code.

Expected Output:

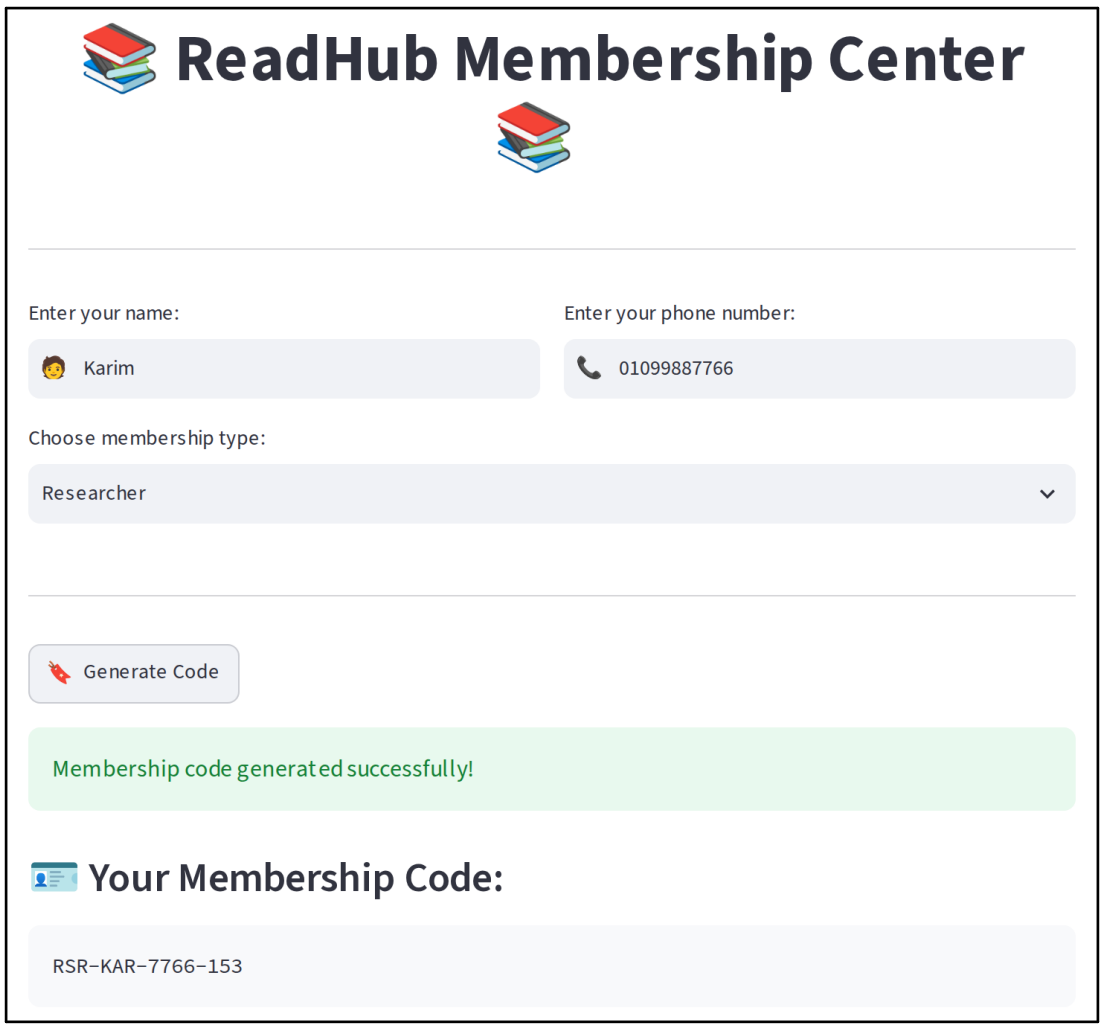
- **The main app layout without the bonus part:**



- **Invalid phone number example:**



• **Another example with the bonus part:**



CODE CHALLENGE CHAMPIONSHIP

Mission 3

CodeChamps - Onsite Practice

Python Junior **Category**

2026

PizzaX Cashier App

Overview:

A new pizza restaurant named **"PizzaX"** wants to help its cashier staff by building an app that quickly calculates the order price. The restaurant offers different cheese and meat pizzas, along with three sizes.

Mission:

1- Create a function named **calculate_order()** that:

- Accepts **2** parameters:
    1. **pizza** → A **string** representing the selected pizza.
    2. **size** → A **string** representing the selected size.
- Uses these prices (the size price is **added** to the pizza price):

Cheese Pizzas	Meat Pizzas	Sizes (added)
Margherita - 100 EGP	Pepperoni - 150 EGP	Small - 0 EGP
Veggie - 120 EGP	BBQ Chicken - 160 EGP	Medium - 30 EGP
Mushroom - 130 EGP	Supreme - 180 EGP	Large - 55 EGP

- Returns **1** value → The **final price** of that pizza.

2- Create the app UI using **Streamlit**:

- Put this **title** → "PizzaX Cashier System" (with pizza emojis).
- Display the menu in **2 columns** as shown in the output section.
- Add a **slider** to let the cashier control the **number of pizzas in the order** (1 to 6).
- Add **2 select boxes** to allow the cashier to choose the **pizza type** and **size** for **each pizza**, as well as displaying the **subtotal**.
- Add a **button** that when clicked, will **call the function** created and display the **total price**.

3- Create a **sidebar** for taking **customer information** containing:

- **2** input fields to take **customer name** and **phone number**.
- A **selectbox** for the **order type** that has **4 options** (Dine-in - Takeaway - Delivery - Drive-thru).
- A **text area** to take the **customer's special requests**.

Bonus Part:

Extend the function by adding a **third parameter order_total**. When it is **greater than 400 EGP**, apply a **15% discount** and return a **second value**; the final price of the order after the discount.

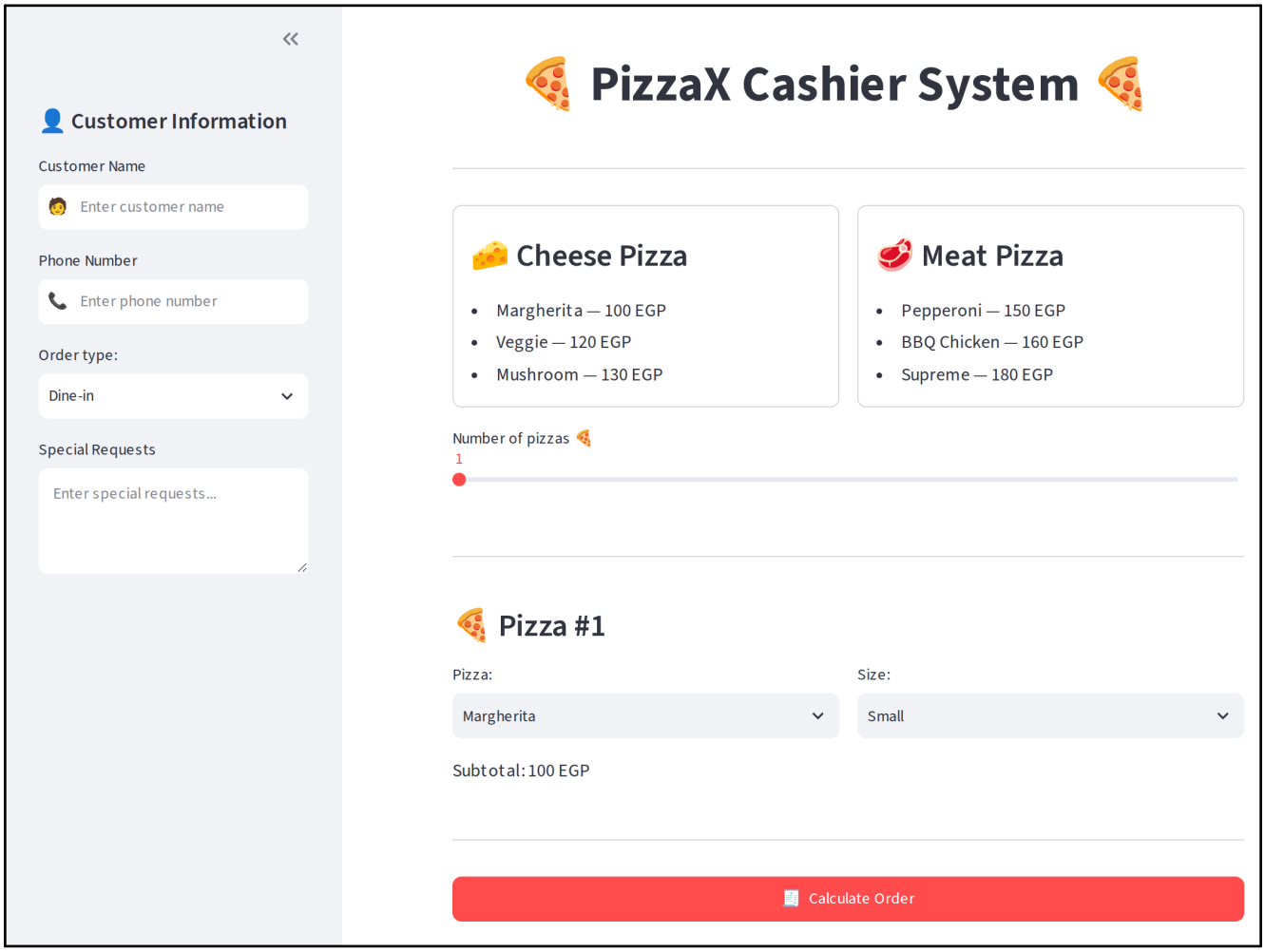
Useful Hints:

- Use a **for loop** to **repeat** the pizza-order section.
- To add a **border** for the columns, use **st.columns(2, border=True)**.
- Use a **unique key** for every widget inside the loop (Ex: key=f'pizza_{i}').

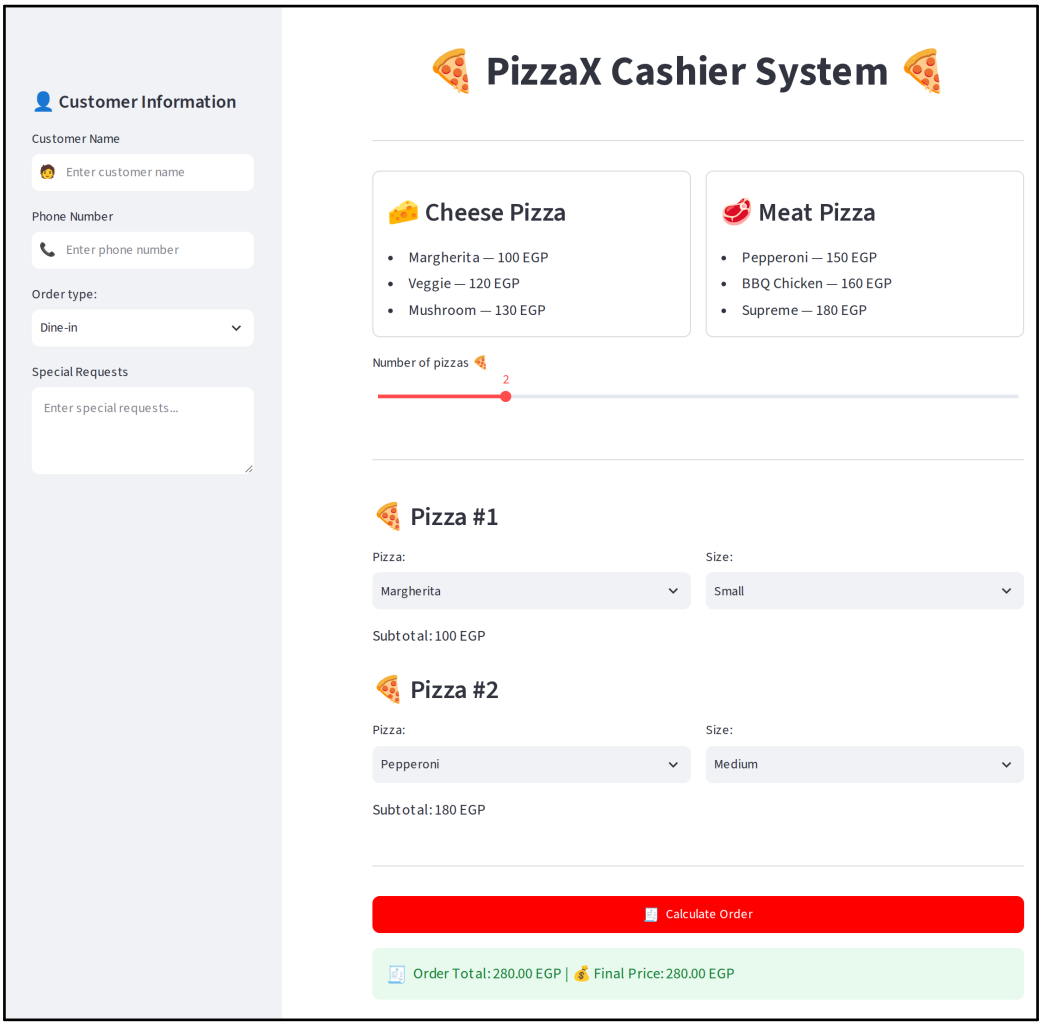
- To make **centered** (title, header, or subheader), add the **text_alignment** parameter with the value 'center'.

Expected Output:

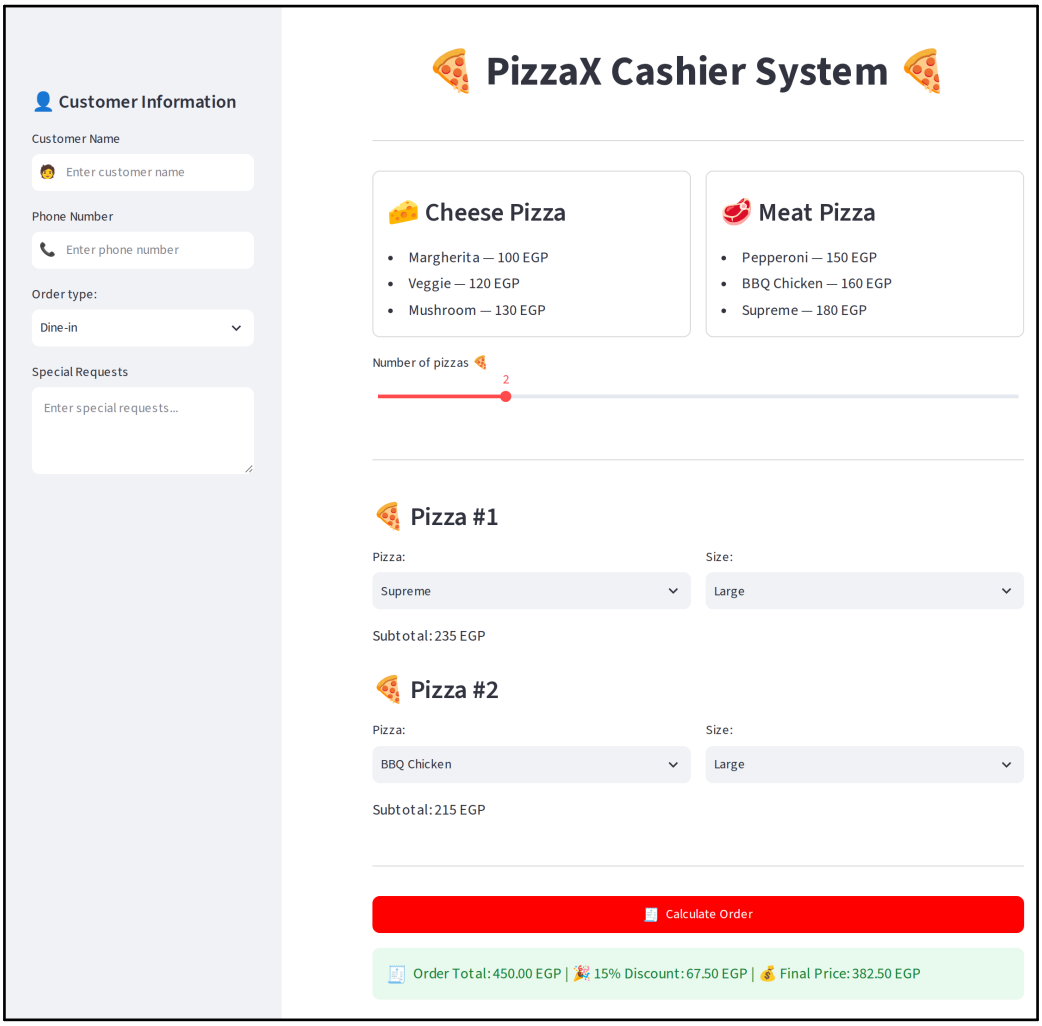
• The main app layout:



• **Ordering 2 pizzas and showing final price with no discount:**



• **Ordering 2 pizzas and showing final price with 15% discount:**



CODE CHALLENGE CHAMPIONSHIP

Mission 3

CodeChamps - Onsite Practice

Python Junior **Category**

2026

CafeX Order System

Overview:

A new coffee shop named **"CafeX"** wants to help its baristas by building an app that quickly calculates the price of each customer's order. The cafe offers hot and cold drinks, along with optional add-ons.

Mission:

1- Create a function named **calculate_order()** that:

- Accepts **2** parameters:
    1. **drink** → A **string** representing the selected drink.
    2. **addon** → A **string** representing the selected add-on.
- Uses these prices:

Hot Drinks	Cold Drinks	Add-ons
Espresso - 45 EGP	Iced Latte - 70 EGP	Extra Shot - 15 EGP
Cappuccino - 60 EGP	Lemonade - 50 EGP	Syrup - 10 EGP
Latte - 65 EGP	Smoothie - 80 EGP	Whipped Cream - 12 EGP
		None - 0 EGP

- Returns **1** value → The **final price** of that drink with its add-on.

2- Create the app UI using **Streamlit**:

- Put this **title** → "CafeX Order System" (with coffee emojis).
- Display the menu in **2 columns** (Hot Drinks and Cold Drinks) with a border.
- Add a **slider** to let the barista control the **number of drinks in the order** (1 to 5).
- Add **2 select boxes** to choose the **drink** and the **add-on** for **each drink**, as well as displaying the **subtotal**.
- Add a **button** that when clicked, will **call the function** created and display the **total price**.

3- Create a **sidebar** for taking **customer information** containing:

- **1** text input for the **customer name** and **1** number input for the **table number** (1 to 20).
- A **selectbox** for the **payment method** that has **4 options** (Cash - Card - Mobile Wallet - Voucher).
- A **text area** to take **special requests** (Ex: less sugar).

Bonus Part:

Extend the function by adding a **third parameter order_total**. When it is **greater than 200 EGP**, apply a **12% discount** and return a **second value**; the final price of the order after the discount.

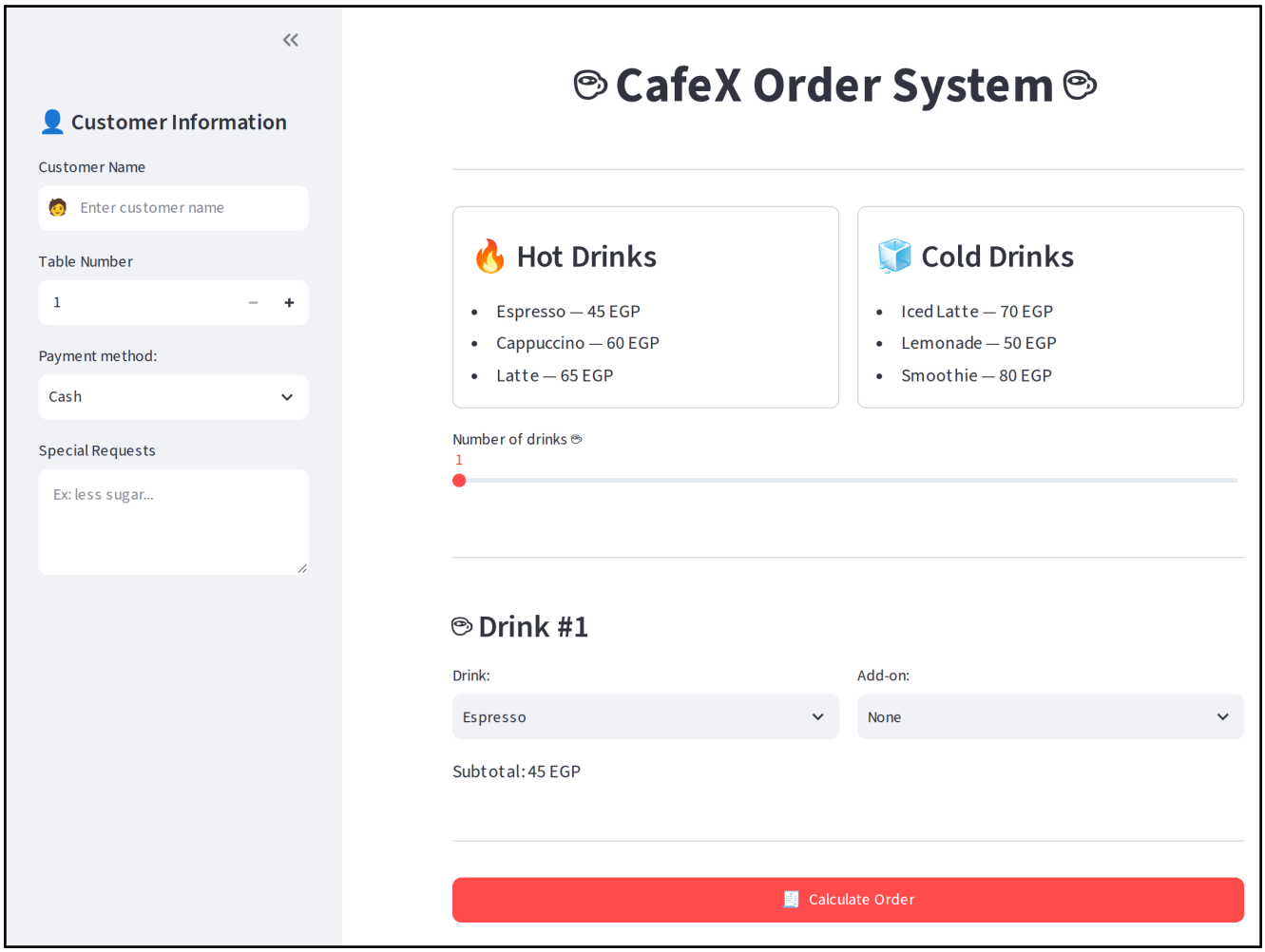
Useful Hints:

- Use a **for loop** to **repeat** the drink-order section and keep the subtotals in a **list**.

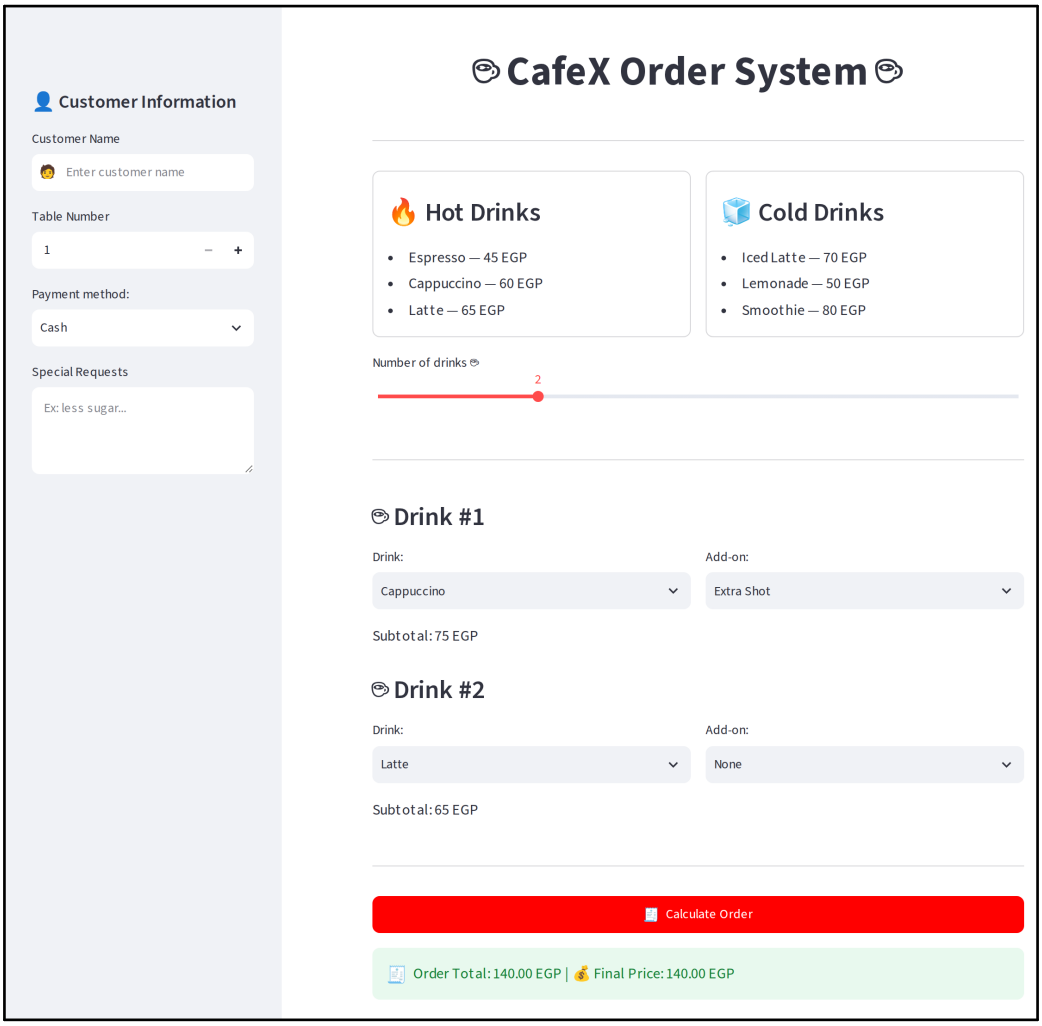
- Use **sum()** on the list of subtotals to get the order total, then pass the info to your function.
- To add a **border** for the columns, use **st.columns(2, border=True)**.
- Use **st.number_input** with **min_value** and **max_value** for the table number.

Expected Output:

• The main app layout:



• **Ordering 2 drinks and showing final price with no discount:**



- **Ordering 3 drinks and showing final price with 12% discount:**

